CDA— Calculadora de Dosagem Agrícola

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	CDA — Calculadora de Dosagem Agrícola
Gerente de Projeto	Lucas Fernando Marins Peixoto
Gerente/Responsável de GCS - DevOps	Lucas Fernando Marins Peixoto
Fábrica de Software / Squad	LP Soluções
Repositório principal	https://github.com/lucaspeixotoo/calculadora-dosagem-agricola

Histórico de Revisões

Data	Versão	Descrição	Autor
26/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Lucas Peixoto

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente.)

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pelo desenvolvedor responsável, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este plano abrange o projeto, controlando e rastreando mudanças no código, implementação de features, pipeline e versionamento para garantir qualidade de consistência ao longo do desenvolvimento.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR. PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado,

testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas sugeridas — substituir pelas efetivamente adotadas pelo projeto:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte e histórico de mudanças
Orquestração de CI/CD	GitHub Actions	Automação de testes a cada push/PR
Empacotamento	Não aplicável	Projeto não gera imagem/container no escopo atual
Registro de artefatos	Não aplicável	-
Infraestrutura como Código	Não aplicável	Projeto não provisiona infraestrutura própria
Orquestração de containers	Não aplicável	-

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Gestão de segredos	Não aplicável	Projeto não utiliza credenciais/segredos no escopo atual
Monitoramento e Observabilidade	Não aplicável	-
Gestão de dependências / SCA	npm	Gerenciamento de pacotes e versões fixadas através do package.json e package-lock.json
Rastreamento de tarefas	GitHub Templates	Uso de arquivos em `.github/ISSUE_TEMPLATE/` e do `pull_request_template.md` para padronizar demandas

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Integração contínua do código de desenvolvedor(es); recriado a cada pipeline.	Merge/push na branch principal (trunk)
Homologação / Staging	Testes de aceitação, integração entre serviços e validação funcional antes da produção.	Pipeline de CI aprovado
Produção	Ambiente utilizado pelos usuários finais.	Aprovação do gate de release (manual ou automática) após validação em staging

3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (.github/workflows/ci.yml)
- Dockerfiles e manifests de containers
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes)
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.
- Scripts de migração de banco de dados (N/A)
- Documentação técnica base (README.md).
- Arquivos de gestão de dependências (package.json e package-lock.json).
- Templates e regras de governança do GitHub (.github/ISSUE_TEMPLATE/, pull_request_template.md e CODEOWNERS).
- Arquivos de configuração de versionamento (.gitignore, .gitattributes).

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/buscar-calculo-por-id
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(api): adiciona rota GET /calculos/:id
Tag de release	Versionamento Semântico vX.Y.Z	v0.1.0
Imagem de container	<registro>/<projeto>:<versão|sha>	ghcr.io/org/projeto:2.3.0
...	...	...

4.3 Estrutura do Repositório

A pasta .github/ centraliza a definição do pipeline de Integração Contínua (ci.yml), as regras de propriedade (CODEOWNERS) e os moldes de projeto (ISSUE_TEMPLATE/ e pull_request_template.md); a pasta docs/ armazena a documentação técnica oficial e os artefatos de gestão do projeto; a pasta src/ contém o código-fonte da aplicação; a pasta tests/ abriga os testes automatizados em Jest; e os arquivos de configuração na raiz (package.json, .gitignore, etc.) gerenciam as dependências e regras do versionamento.



5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código-fonte.
2. Restauração e instalação de dependências via gerenciador de pacotes (npm install).
3. Execução da suíte de testes automatizados (unitários e de integração) através do comando configurado no projeto (npm test).

5.2 Política de Pull Request e Revisão de Código

A qualidade do código é assegurada através da revisão por pares (Peer Review). Todo Pull Request exige no mínimo uma (1) aprovação de outro desenvolvedor (revisor) da equipe antes do merge. Os PRs devem ser mantidos pequenos e focados em um único escopo (rastreados via template de PR) para facilitar a análise. O merge para a branch principal é estritamente bloqueado até que o revisor aprove as alterações e o pipeline de CI (Quality Gate) retorne sucesso (verde).

5.3 Critérios de Qualidade (Quality Gate)

Todos os testes automatizados devem passar (pipeline de CI verde) antes do merge. Cobertura de testes mínima ainda não definida; planejada como melhoria do processo.

6. Entrega e Implantação Contínua (CD)

Nota de escopo: as etapas abaixo descrevem o fluxo de CD completo, incluindo ambientes de produção. No estado atual do projeto (focado na integração contínua colaborativa e sem deploy automatizado para um servidor de produção real), o CD limita-se à formalização da baseline/tag de versão após o merge na branch principal, conforme detalhado na seção 9. As etapas de staging/produção/rollback aqui descritas representam o processo de referência a ser adotado caso o projeto evolua para um cenário com implantação em infraestrutura real em nuvem.

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

4. Deploy automático em ambiente de Homologação/Staging.
5. Execução de testes automatizados de aceitação/E2E em staging.
6. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
7. Deploy em produção com estratégia definida (ver 6.2).
8. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
9. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. O pipeline de CD deve manter a versão anterior disponível (imagem no registro, IaC versionada) para permitir rollback automatizado em caso de falha nos testes de fumaça ou nos indicadores de saúde pós-deploy.

7. Infraestrutura como Código (IaC)

Nota de escopo: este projeto não provisiona infraestrutura própria (sem containers, sem cloud, sem Terraform), conforme já indicado na seção 3.1. Esta seção é mantida no documento como referência do

processo a ser adotado caso o escopo evolua para incluir infraestrutura provisionada.

Toda a infraestrutura provisionada para os ambientes de Desenvolvimento, Homologação e Produção deve ser descrita como código versionado, submetida ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

7.1 Versionamento de Infraestrutura

- Alterações de infraestrutura seguem o mesmo processo de PR + revisão + pipeline ("plan" automático em PR, "apply" após aprovação).
- O estado da infraestrutura (state file) é armazenado de forma centralizada e versionada, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Sempre que viável, ambientes de revisão (review apps) devem ser provisionados automaticamente por Pull Request, para validação isolada, e destruídos automaticamente após o merge ou fechamento do PR.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Devido ao tamanho enxuto da equipe e à agilidade do projeto, não há instituição de um Comitê de Controle de Mudança (CCM) formal e burocrático. O controle de impacto e as decisões sobre alterações estruturais são gerenciados de forma distribuída através da revisão de código. Toda mudança exige abertura de Issue (utilizando os templates padronizados do repositório) e submissão via Pull Request. A aprovação do revisor somada à validação automatizada do pipeline de CI formam a barreira de aprovação definitiva para a integração de qualquer mudança.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no registro e ao estado da infraestrutura correspondente.

9.2 Notas de Versão (Release Notes)

As notas de release são elaboradas manualmente a partir do histórico de commits e documentadas diretamente nas tags de versão (releases) do repositório no GitHub, uma vez que a geração automatizada via ferramentas de versionamento semântico não está implementada no escopo atual.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção devem ser monitorados continuamente (métricas, logs e alertas), permitindo identificar rapidamente falhas decorrentes de uma implantação.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

- Estruturação inicial do projeto (rotas, lógica de cálculo, testes) e primeiro commit publicado na branch `main`
- Configuração do pipeline de Integração Contínua via GitHub Actions
- Execução do primeiro ciclo completo de mudança: branch de feature → Pull Request → pipeline de CI aprovado → merge na `main`
- Publicação da primeira baseline de versão (`v0.1.0`) via tag Git
- Execução e validação do primeiro ciclo colaborativo de mudança: criação de branch de feature → abertura de Pull Request (utilizando o template padronizado) → **aprovação via revisão por pares (Peer Review)** → sucesso no pipeline de CI → merge na branch principal.

12. Treinamento e Recursos

A governança de conhecimento e o alinhamento da equipe dispensam cronogramas formais de treinamento, baseando-se fortemente na padronização via documentação técnica e na troca de conhecimento durante as revisões de código (Peer Review). Os recursos essenciais para o nivelamento da equipe fundamentam-se na leitura e aplicação da documentação oficial das tecnologias vigentes (GitHub Actions, Jest, Node.js) e na adoção disciplinada de padrões de mercado, como o Conventional Commits para rastreabilidade do histórico e o Semantic Versioning para o controle de releases.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

O projeto não envolve subcontratados ou fornecedores externos; todas as dependências são bibliotecas públicas de terceiros gerenciadas via npm. As versões utilizadas são fixadas de forma determinística no

arquivo package-lock.json. No escopo atual, a verificação de vulnerabilidades e a atualização de dependências são realizadas de forma manual, não havendo processos automatizados (como Dependabot ou ferramentas de SCA) integrados ao pipeline de CI.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de segurança, publicação do artefato/imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Staging	Deploy automático em homologação + testes de aceitação.	Merge na branch principal
Gate de Release	Aprovação para produção (automática ou manual, conforme criticidade).	Sucesso dos testes em staging
CD — Produção	Deploy (rolling/blue-green/canary), smoke tests, geração de baseline/tag.	Aprovação do gate de release

Nota: *este template deve ser adaptado ao contexto real do projeto (ferramentas, criticidade do sistema e maturidade da equipe em DevOps), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação.*